

4.1 / 4.2

Overview of Network Layer · What's Inside a Router?

Kurose & Ross, 8th Ed.



Chapter 4 — Network Layer: Data Plane

Introduction

The network layer is the most complex layer — it runs in EVERY host and EVERY router in the Internet

Why Two Chapters?

Chapter 4 — DATA PLANE:

How a single router decides which output link to send each arriving packet (per-router, hardware, nanoseconds).

Chapter 5 — CONTROL PLANE:

How routers collectively compute end-to-end paths; routing protocols OSPF and BGP; SDN controllers.

Today's Lecture

4.1 Overview of Network Layer:

- Forwarding vs. Routing
- Data plane vs. Control plane
- Per-router vs. SDN control
- Network service model

4.2 What's Inside a Router?:

- Input ports, switching fabric, output ports, routing processor
- Longest prefix matching, TCAM
- Queuing: HOL blocking, output queuing
- Packet scheduling: FIFO, Priority, WFQ

Application

Transport

Network ← We are here

Link + Physical

Unlike the transport and application layers (end systems only), network-layer protocols run in EVERY host and EVERY router

4.1.1 Forwarding vs. Routing

Section 4.1: Overview of Network Layer

The network layer's primary role: move packets from a sending host to a receiving host

FORWARDING (Data Plane)

When a packet arrives at a router's input link, move it to the APPROPRIATE OUTPUT LINK.

This is a LOCAL, per-router action.

Analogy: getting through a SINGLE INTERCHANGE — a car enters from one road and the router sign tells it which exit to take.

Timescale: NANOSECONDS

Implementation: HARDWARE (ASICs)

ROUTING (Control Plane)

Determine the ROUTE or PATH taken by packets from a sender to a receiver across the entire network.

This is a NETWORK-WIDE process.

Analogy: planning the WHOLE TRIP from Pennsylvania to Florida — consulting a map and choosing among many paths before embarking.

Timescale: SECONDS

Implementation: SOFTWARE

KEY ELEMENT in every router: FORWARDING TABLE — maps header values to output link. Populated by the ROUTING ALGORITHM.

Data Plane vs. Control Plane

Section 4.1.1

Forwarding ↔ Data Plane · Routing ↔ Control Plane

DATA PLANE

- Per-router **HARDWARE** that actually moves packets.
- Examines packet header, looks up forwarding table, sends to output port.
- Nanosecond timescale.
- Implemented in custom ASICs or merchant silicon (Intel, Broadcom).
- Runs **INDEPENDENTLY** at every router simultaneously.
- 100 Gbps link: 5.12 ns per 64-byte datagram — no time for software.

CONTROL PLANE

- **NETWORK-WIDE LOGIC** that determines how forwarding tables are built.
- Computes end-to-end paths packets should follow.
- Second timescale.
- Implemented in **SOFTWARE** on conventional CPUs.
- Two architectural approaches:
 - Per-router (traditional)
 - Logically centralized (SDN)
- Examples: OSPF, BGP, OpenFlow controller.

Two Control Plane Approaches

Section 4.1.1

How are forwarding tables computed and installed — in each router, or by a remote controller?

Per-Router Control (Traditional)

A routing algorithm runs in EACH AND EVERY router.
Both forwarding AND routing contained inside each router.

Routing component in each router communicates with routing components in other routers — exchanging ROUTING MESSAGES — to collectively compute forwarding tables.

Per-router control has been used in the Internet for DECADES.

Protocols: OSPF (intra-domain) · BGP (inter-domain)

Advantages:

- Distributed — no single point of failure
- Proven, widely deployed

Logically Centralized Control (SDN)

A physically SEPARATE, REMOTE CONTROLLER computes forwarding tables and DISTRIBUTES them to every router.

Routers are "dumb" forwarding devices — no routing algorithms inside.

Controller has GLOBAL VIEW of entire network.
Communicates with routers via OpenFlow protocol.

Advantages:

- PROGRAMMABLE — change routing by modifying software
- GLOBALLY OPTIMAL decisions
- Separation of hardware (vendors) and software (operators)

Deployments:

- Google B4 — internal data-center WAN
- AT&T: 75% of network functions virtualized

Both approaches use forwarding tables at routers — what differs is WHERE and HOW those tables are computed

4.1.2 Network Service Model — Best-Effort IP

Section 4.1: Overview of Network Layer

The service model defines what the network layer **PROMISES** to deliver end-to-end

Internet's IP Service Model: BEST-EFFORT

- NO guarantee of packet delivery — packets can be lost
- NO guarantee of in-order delivery — packets can arrive out of sequence
- NO guarantee of timing — no bound on end-to-end delay
- NO guarantee of minimum throughput — the network may be slow
- NO guarantee of security — datagrams can be read or modified

Why Best-Effort? The End-to-End Argument

Saltzer, Reed & Clark (1984): don't implement a function in the network if it can be done better in the end systems.

Best-effort makes the network core SIMPLE and CHEAP.

Complexity is pushed to the EDGE:

- TCP provides reliability on top of unreliable IP
- Applications handle timing and ordering

This simplicity enables GLOBAL SCALE:

- Works over any physical medium
- No per-flow state in routers
- Scales to billions of hosts

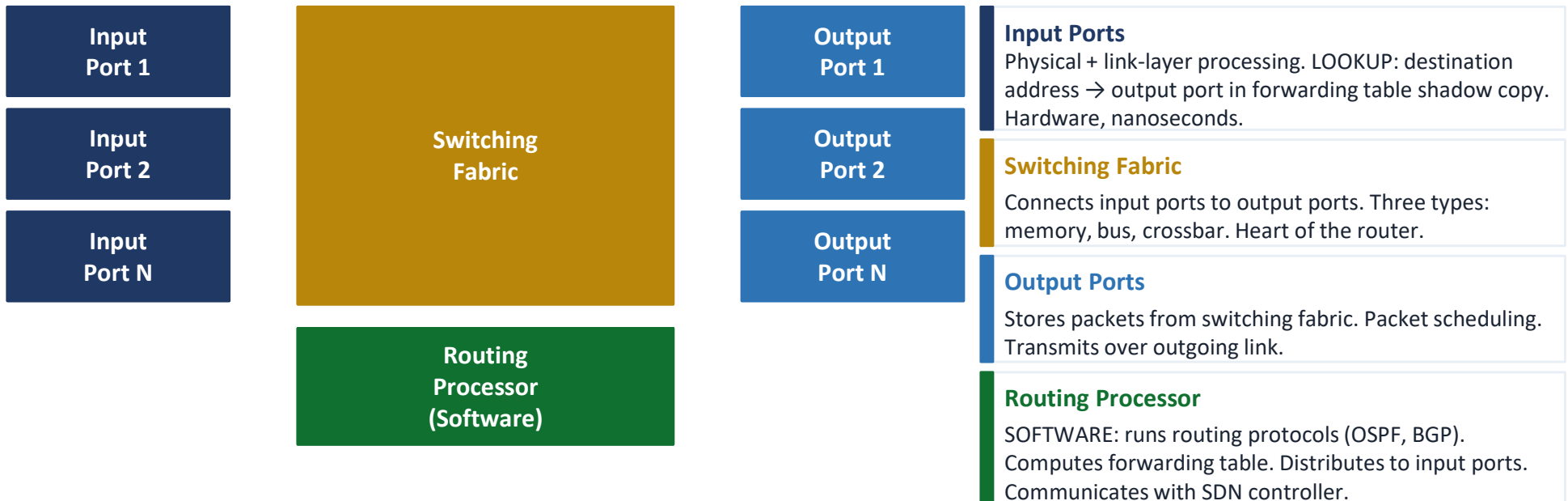
Alternatives (ATM): guaranteed bandwidth, bounded delay, in-order delivery — but none achieved Internet scale.

Datagram name: the network-layer packet is called a DATAGRAM.

4.2 What's Inside a Router? — Architecture Overview

Section 4.2

Four key components: input ports · switching fabric · output ports · routing processor



Input ports, switching fabric, output ports = **HARDWARE** (data plane) · Routing processor = **SOFTWARE** (control plane)

4.2.1 Input Port Processing

Section 4.2: What's Inside a Router?

Three pipeline stages at every input port — all implemented in hardware at nanosecond speed

1 Physical Layer

Terminates the incoming physical link.
Converts optical signal or electrical signal to bits.
Clock recovery, bit-level error detection.

2 Link Layer

Interoperates with the link layer on the other side of the link.
Processes Ethernet frames, decapsulates datagram.
Computes/verifies link-layer checksum.

3 Lookup & Forward

MOST CRUCIAL: looks up destination IP address in the SHADOW COPY of the forwarding table.
Determines output port.
Sends packet into switching fabric.
Control packets → routing processor.

Why a SHADOW COPY of the forwarding table at each input port?

On a 100 Gbps link, a 64-byte IP datagram arrives every 5.12 NANOSECONDS. N ports on one line card = $N \times$ faster pipeline needed. There is simply no time to contact a central forwarding table — lookup must be done IN HARDWARE, LOCALLY. The routing processor updates all shadow copies whenever the forwarding table changes.

"Match plus action" abstraction: match destination address → action: forward to output port. Same abstraction used in firewalls, NAT, SDN (OpenFlow).

Destination-Based Forwarding & Longest Prefix Match

Section 4.2.1

Forwarding decision based **SOLELY** on the destination IP address — lookup in the forwarding table

Forwarding table (prefix → output link):

Destination prefix	Link
11001000 00010111 00010 (21 bits)	0
11001000 00010111 00011000 (24 bits) ←	1
11001000 00010111 00011 (21 bits)	2
otherwise (default route)	3

Example: destination = 11001000 00010111 00011001 00001011

Matches prefix 0 (21 bits) AND prefix 2 (21 bits). Longest match rule → use the LONGEST matching prefix → link 2.

TCAM — Ternary Content-Addressable Memory

Present a 32-bit IP address → memory returns the matching forwarding entry in **CONSTANT TIME** (one memory cycle). No linear search needed. Modern routers: up to **MILLIONS** of TCAM entries. Cisco Catalyst 6500/7600: upwards of 1 million entries.

4.2.2 Switching Fabric — Memory and Bus

Section 4.2: What's Inside a Router?

The switching fabric is the heart of the router — it physically moves packets from input to output ports

Switching via Memory (1st generation)

The SIMPLEST, EARLIEST approach.

Process:

1. Packet arrives at input port → sends interrupt to routing processor (CPU)
2. CPU copies packet from input port into PROCESSOR MEMORY
3. CPU extracts destination address, looks up forwarding table
4. CPU copies packet to output port's buffer

Limitation: SHARED SYSTEM BUS — only ONE memory read/write at a time.
Even two packets headed to different outputs must be processed SEQUENTIALLY.

If memory bandwidth = B packets/sec:
Total throughput $\leq B/2$
(each packet: one read + one write)

Some modern routers still use this approach with very fast memory.

Switching via Bus (2nd generation)

Input ports transfer packets to output ports via a SHARED BUS — routing processor not involved in packet forwarding.

All input and output ports are connected to a SINGLE bus.
Packet is broadcast: ALL output ports see it, but only the DESIGNATED output port keeps it (based on a label added by the input port).

Limitation: only ONE packet can cross the bus AT A TIME.
Packets from different input ports must CONTENTEND for the shared bus — they wait even if their destinations don't conflict.

Throughput limited by bus bandwidth.
Typical: 32 Gbps (Cisco 5600 series).
Suitable for access and enterprise routers.

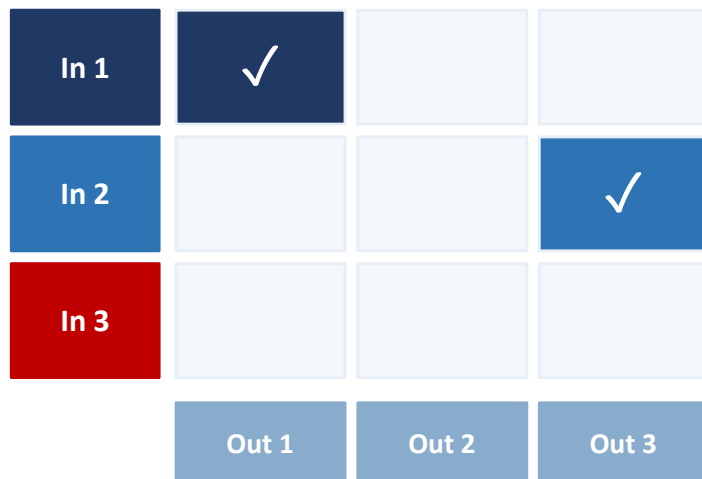
Both memory and bus switching are fundamentally SEQUENTIAL — only one packet at a time. High-speed routers need the crossbar.

Switching via Interconnection Network — Crossbar

Section 4.2.2

Crossbar switches overcome the bandwidth limitation of a single shared bus — multiple packets in PARALLEL

Crossbar: $N \text{ inputs} \times N \text{ outputs} = 2N \text{ buses}$



In1→Out1 and In2→Out3 simultaneously! ✓

Crossbar Properties

Architecture: $N \text{ input ports} \times N \text{ output ports}$ connected by $2N$ buses. Each intersection = a CROSSPOINT (open/closed by switch controller).

PARALLEL FORWARDING: multiple packets can be forwarded SIMULTANEOUSLY as long as they go to DIFFERENT output ports.

NON-BLOCKING: a packet headed to output Y is never blocked by packets headed elsewhere — UNLESS another packet is simultaneously heading to the same output Y.

If $R_{\text{switch}} = N \times R_{\text{line}}$: all N input ports can forward simultaneously with NO input queuing.

More advanced: MULTI-STAGE fabrics (Cisco CRS: 3-stage non-blocking). PARALLEL FABRICS: input port sprays packet chunks across K fabrics; output port reassembles.

Examples:

- Cisco 12000: crossbar switching network
- Juniper MX2020: up to 800×100 Gbps ports, 800 Tbps total

Crossbar is the dominant switching fabric in all high-speed routers — it enables true parallel forwarding

Switching Fabrics — Summary Comparison

Section 4.2.2

Three generations of switching fabric design — each overcomes the bottleneck of the previous

Dimension	Via Memory	Via Bus	Via Crossbar
Packet path	Input → CPU memory → Output	Input → Shared bus → Output	Input → Crosspoint → Output
CPU involved?	Yes (interrupt per packet)	No	No
Parallel forwarding?	No (1 at a time)	No (1 at a time)	YES (different outputs)
Bottleneck	Memory bandwidth $B/2$	Bus bandwidth	Same output port contention
Throughput limit	$B/2$ packets/sec	Bus BW (e.g., 32 Gbps)	$N \times R_{line}$ (scales with N)
Timescale	μsec (memory access)	Sub- μsec	Nanoseconds
Typical use	Early/simple routers	Access/enterprise	High-speed backbone/ISP
Examples	Classic routers	Cisco 5600	Cisco 12000, Juniper MX

4.2.4 Where Does Queuing Occur?

Section 4.2: What's Inside a Router?

Packets can queue — and be LOST — at BOTH input ports and output ports

Setup: N input/output ports · Line rate R_{line} packets/sec · Fabric rate R_{switch} · Packets same size

Output Queuing (when $R_{\text{switch}} = N \times R_{\text{line}}$)

Worst case: ALL N input ports simultaneously receive one packet, ALL destined for the SAME output port.

Fabric ($N \times$ faster) delivers all N to the output in one time slot.
Output port can only transmit ONE per time slot.
→ $N-1$ packets queue at the output port.

N more packets may arrive in the next slot → queue grows.

OUTPUT QUEUING IS INEVITABLE even with a fast fabric when multiple packets target the same output.

Packet loss occurs when output buffer is full.

Input Queuing (when $R_{\text{switch}} < N \times R_{\text{line}}$)

When the fabric is too SLOW to clear arriving packets before the next batch arrives, packets must queue at INPUT PORTS.

Consequence: HEAD-OF-LINE (HOL) BLOCKING — a queued packet at the HEAD of an input queue prevents packets BEHIND it from moving, even if those packets' output ports are FREE.

HOL blocking limits throughput to $\approx 58\%$ of line rate!
(Karol et al., 1987)

Solution: VIRTUAL OUTPUT QUEUEING (VOQ) — maintain a separate queue at each input for EACH output port.

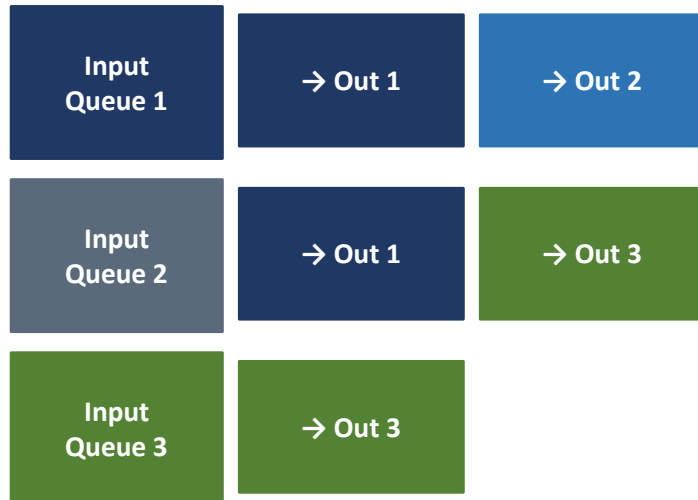
Packets dropped and lost always occur at QUEUES within a router — this is where "lost within the network" actually happens

Head-of-Line (HOL) Blocking — A Critical Bottleneck

Section 4.2.4

HOL blocking: a packet must wait in an input queue even though its output port is FREE

HOL Blocking Scenario (crossbar, 3 inputs)



Scheduler picks: Q1→Out1 ✓ and Q3→Out3 ✓
Q2 head packet (→Out1) BLOCKED. But Q2's 2nd packet (→Out3) is also blocked!
Out3 is FREE — but the 2nd packet CANNOT MOVE past the 1st.

Impact & Solutions

IMPACT:

Karol et al. (1987) proved: due to HOL blocking, input queue grows to unbounded length (= severe packet loss) as soon as arrival rate reaches only 58% of line capacity.

→ Maximum useful throughput of an input-queued switch = ~58% of line rate
 → Even with a perfect crossbar!

This was a fundamental result that changed router design.

SOLUTIONS:

1. SPEEDUP: Make $R_{\text{switch}} \gg R_{\text{line}}$ (so queuing moves entirely to output ports, where HOL cannot occur). Common in practice.
2. VIRTUAL OUTPUT QUEUEING (VOQ):
 Each input port maintains a SEPARATE queue per output port.
 Packet for Out2 never blocks packet for Out3 in the same input.
 Combined with iSLIP scheduling algorithm.
 Eliminates HOL blocking completely.

Output Port Queuing and Buffer Sizing

Section 4.2.4

Output port queuing is inevitable — a packet scheduler must choose which packet to send next

Output Port Queuing Dynamics

At time t : N packets arrive at N input ports, all destined for the SAME output port.

Fabric ($N\times$ faster) delivers all N to output in 1 slot.

Output can only TRANSMIT 1 per slot.

→ $N-1$ packets wait in output queue.

At $t+1$: 1 transmitted, N more may arrive → queue now $2N-1$.

When buffer is full:

- DROP-TAIL: drop newest arriving packet (default)
- AQM (RED, PIE, CoDel): drop/mark BEFORE buffer is full to signal congestion early to TCP senders

How Much Buffering Is Enough?

OLD rule of thumb (RFC 3439, Villamizar 1994):

$$B = RTT \times C$$

($RTT \approx 250$ ms, C = link capacity)

Example: 10 Gbps link → $B = 2.5$ Gbits

MODERN research (Appenzeller et al., SIGCOMM 2004):

$$B = RTT \times C / \sqrt{N}$$

(N = number of independent TCP flows)

For large N , much LESS buffering is needed.

Core routers with $N=10,000$ flows: B reduced by $100\times$

BUFFERBLOAT: Too much buffering causes enormous queuing delays for real-time applications!

Large buffers absorb TCP congestion signals — TCP senders don't slow down. Queuing delay can reach hundreds of ms even with no packet loss. VoIP, gaming, and video calls become unusable.

Modern AQM algorithms (CoDel, PIE) deliberately KEEP QUEUES SHORT — they drop/mark packets early to control delay, not just loss. Now standard in Linux kernels and home routers.

4.2.5 Packet Scheduling — FIFO and Priority

Section 4.2: What's Inside a Router?

When multiple packets are queued at an output port, which one gets transmitted first?

FIFO / FCFS Scheduling

Packets transmitted in the ORDER THEY ARRIVE.

First-In-First-Out: the oldest packet in the queue goes first.

Advantages:

- Extremely simple to implement
- Fair in terms of arrival order

Disadvantages:

- No differentiation between traffic types
- A single large file transfer can swamp VoIP packets
- DROP-TAIL when full: newest arriving packet is dropped

When FIFO is full and a VoIP packet arrives, it is dropped even though it is much more delay-sensitive than the queued bulk-transfer packets.

FIFO is still the most widely used scheduling policy by volume.

Priority Queuing

Incoming packets classified into PRIORITY CLASSES.

The scheduler ALWAYS serves the highest-priority non-empty class first. Lower classes served ONLY when all higher classes are empty.

Example (3 classes):

- Priority 1: VoIP (real-time, delay-sensitive)
- Priority 2: HTTP web traffic
- Priority 3: Bulk file transfer (FTP)

As long as VoIP packets arrive, HTTP and FTP wait.

Advantage: delay-sensitive traffic always gets priority.

Risk: STARVATION — if high-priority traffic is heavy, low-priority queues may NEVER be served.

In practice: traffic POLICING limits high-priority traffic volume to prevent starvation.

FIFO: simple, no differentiation · Priority: delay-sensitive traffic wins, but risk of starvation

Packet Scheduling — Round Robin and WFQ

Section 4.2.5

WFQ provides proportional bandwidth guarantees — the dominant scheduling algorithm in modern routers

Round Robin (RR) Scheduling

Packets classified into CLASSES. Scheduler serves one packet from each non-empty class in a ROTATING ORDER.

Example (2 classes A and B):

Service order: A, B, A, B, A, B ...

If one class is empty, scheduler skips to the next.

No class can STARVE another.

All classes get EQUAL SERVICE over time — each gets $1/N$ of the bandwidth where N = number of non-empty classes.

Limitation: what if some classes NEED MORE bandwidth?

VoIP needs 10% of link capacity

HTTP needs 60%

File transfer gets the rest

Equal shares are the wrong allocation!

Round Robin treats all classes equally even when they have very different needs.

Weighted Fair Queuing (WFQ)

Generalization of Round Robin — each class assigned a WEIGHT w_i .

Service proportional to weights:

Class 1: $w = 0.5 \rightarrow$ gets 50% of bandwidth

Class 2: $w = 0.25 \rightarrow$ gets 25% of bandwidth

Class 3: $w = 0.25 \rightarrow$ gets 25% of bandwidth

Over a 8-packet period: class 1 gets 4 served, classes 2 and 3 get 2 each.

Each class guaranteed AT LEAST its proportional share, REGARDLESS of other classes.

Bounds MAXIMUM QUEUING DELAY for each class (unlike FIFO).

Differentiated Services (DiffServ, RFC 2597): WFQ is the implementation mechanism that delivers per-class QoS guarantees.

DEPLOYMENT: WFQ is the dominant scheduling algorithm in modern backbone routers. Used in enterprise QoS policies, ISP traffic engineering, and cellular base stations.

WFQ: proportional bandwidth · bounded delay · no starvation · foundation of network QoS guarantees

Packet Scheduling — Summary and Practical Example

Section 4.2.5

Scheduling matters: the wrong policy can make real-time apps unusable even with plenty of bandwidth

Policy	How it works	Starvation?	BW Guarantee?	Best for
FIFO / Drop-tail	Serve in arrival order; drop newest when full	No	No	Simple networks, best-effort only
Priority Queuing	Always serve highest non-empty class first	YES	No	Delay-sensitive + good traffic policing
Round Robin (RR)	One packet per class, rotating	No	Equal shares	Equal-importance classes
WFQ	Weighted round-robin — proportional service	No	YES (proportional)	QoS, DiffServ, modern ISPs

Practical Example: Home Router (Zoom call + large file download)

With FIFO:

The file download (20 Mbps) occupies most of the output link. Zoom packets (2 Mbps) wait behind large file-transfer packets. Result: choppy video, degraded audio quality. Bufferbloat makes this even worse — long queues in the router.

With WFQ (Zoom weight = 0.6, Download = 0.4):

Zoom packets get 60% of link bandwidth — served promptly. File download gets remaining 40% — still fast. Result: smooth video, good audio AND fast download. This is exactly why modern QoS routers implement WFQ.

Complete Router Architecture — Data vs. Control Plane

Section 4.2: Summary

Hardware (data plane): nanosecond forwarding · **Software (control plane): second-scale routing**

INPUT PORT (Hardware)

- Physical + link-layer processing
- Shadow copy of forwarding table
- Destination-based lookup: LONGEST PREFIX MATCH
- TCAM: constant-time lookup at Gbps speeds
- Queuing possible (→ HOL blocking if fabric slow)
- Control packets → routing processor

OUTPUT PORT (Hardware)

- Stores packets from switching fabric
- Packet scheduling: FIFO / Priority / RR / WFQ
- Output queuing inevitable: N packets → same port
- Packet loss: DROP-TAIL, RED, CoDel (AQM)
- Buffer sizing: $B = RTT \times C / \sqrt{N}$ (modern)

SWITCHING FABRIC (Hardware)

- Via MEMORY: simplest, sequential, throughput $\leq B/2$
- Via BUS: shared, limited BW, no CPU involved
- Via CROSSBAR: parallel, non-blocking (diff. outputs)
- Multi-stage fabrics for Tbps backbone routers
- $R_{\text{switch}} = N \times R_{\text{line}} \rightarrow$ negligible input queuing

ROUTING PROCESSOR (Software)

- Executes routing protocols: OSPF, BGP
- Maintains routing tables
- Computes the forwarding table
- Distributes shadow copies to all input ports
- Communicates with SDN controller (OpenFlow)

Summary

4.1 / 4.2 — Network Layer Overview · Router Architecture

- Network layer runs in EVERY host and router. Primary role: move datagrams from source to destination host.
- FORWARDING: local, per-router, hardware, nanoseconds — moves packet from input to output link. ROUTING: network-wide, software, seconds — computes end-to-end paths.
- DATA PLANE: hardware (input ports, fabric, output ports) — the per-router forwarding mechanism. CONTROL PLANE: software (routing processor) — computes forwarding tables.
- Per-router control (OSPF/BGP): routing algorithm in each router. SDN (OpenFlow): remote controller computes and distributes forwarding tables.
- Network service model: BEST-EFFORT IP — no guarantees of delivery, order, timing, or throughput. Simplicity enables global scale (End-to-End Argument).
- Router components: input ports (lookup + shadow table), switching fabric (memory/bus/crossbar), output ports (scheduling), routing processor (software).
- Longest prefix matching: when multiple prefixes match, use the MOST SPECIFIC. TCAM provides constant-time lookup at Gbps speeds.
- Switching fabrics: memory (B/2 limit, sequential) → bus (shared, limited BW) → crossbar (parallel, non-blocking, dominant in high-speed routers).
- HOL blocking: input-queued switch throughput capped at ~58% (Karol 1987). Solution: virtual output queuing (VOQ) or speedup.
- Packet scheduling: FIFO (simple) → Priority (starvation risk) → Round Robin (equal shares) → WFQ (weighted proportional BW — dominant in modern routers).